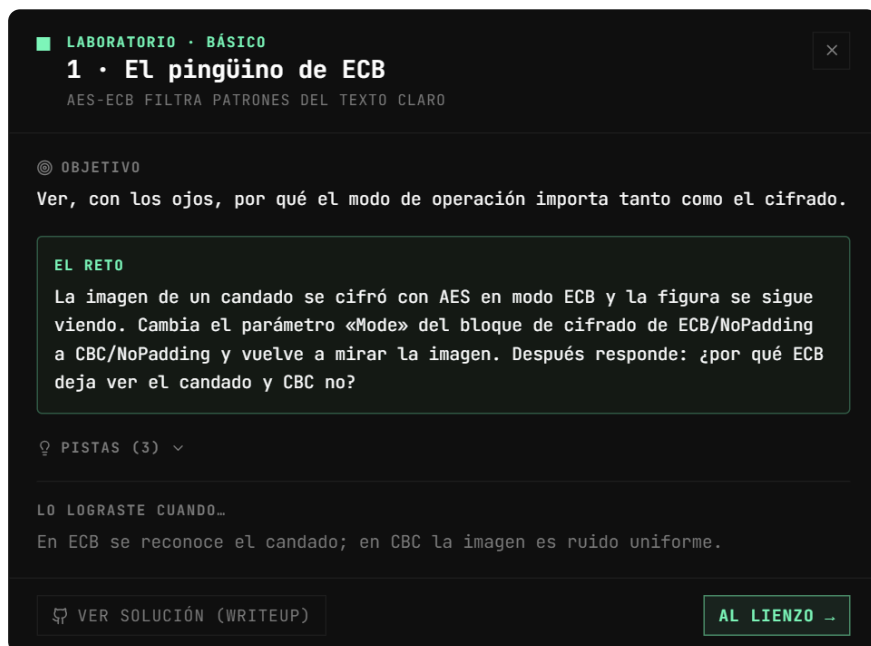


Lab 1 — El pingüino de ECB

Vulnerabilidad: ECB (Electronic Codebook) cifra cada bloque de 16 bytes de forma independiente y determinista. Dos bloques de texto claro iguales producen dos bloques cifrados iguales, así que la estructura del mensaje sobrevive al cifrado. El nombre viene del clásico experimento de cifrar la imagen del pingüino de Linux (Tux): tras ECB, la silueta sigue siendo visible.

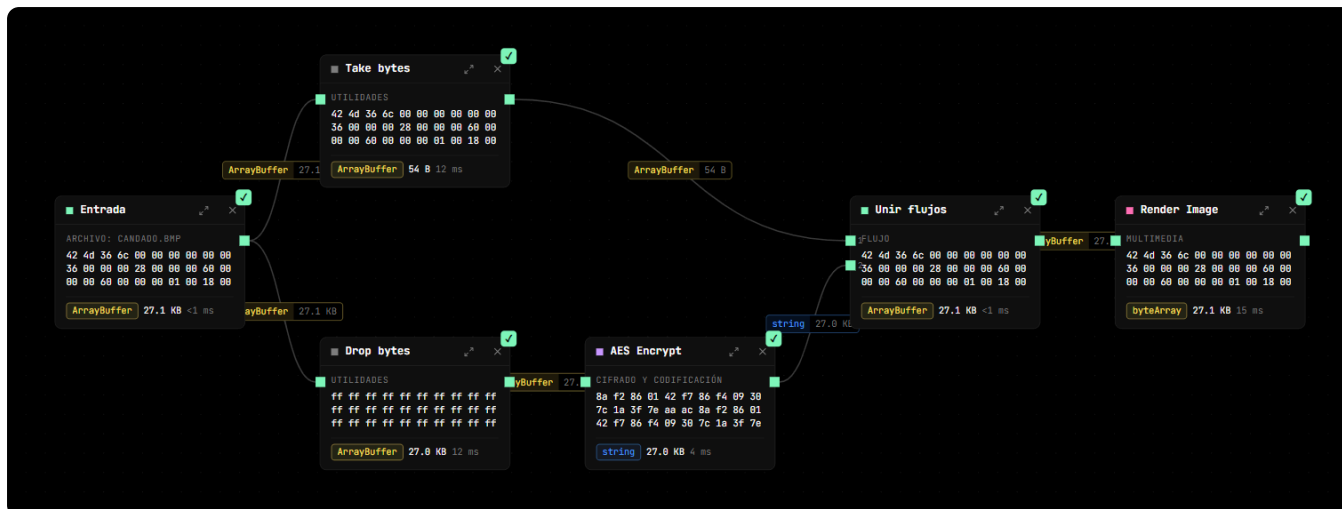
El reto

La imagen de un candado se cifró con AES-ECB y la figura se sigue viendo. Cambia el modo a CBC y explica por qué desaparece.

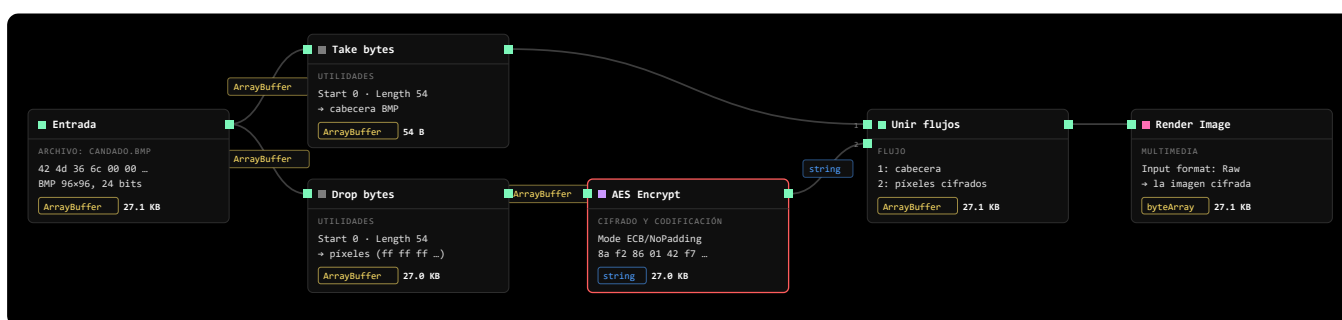


Solución

El flujo ya viene montado al elegir **Laboratorios** → **1 · El pingüino de ECB**:



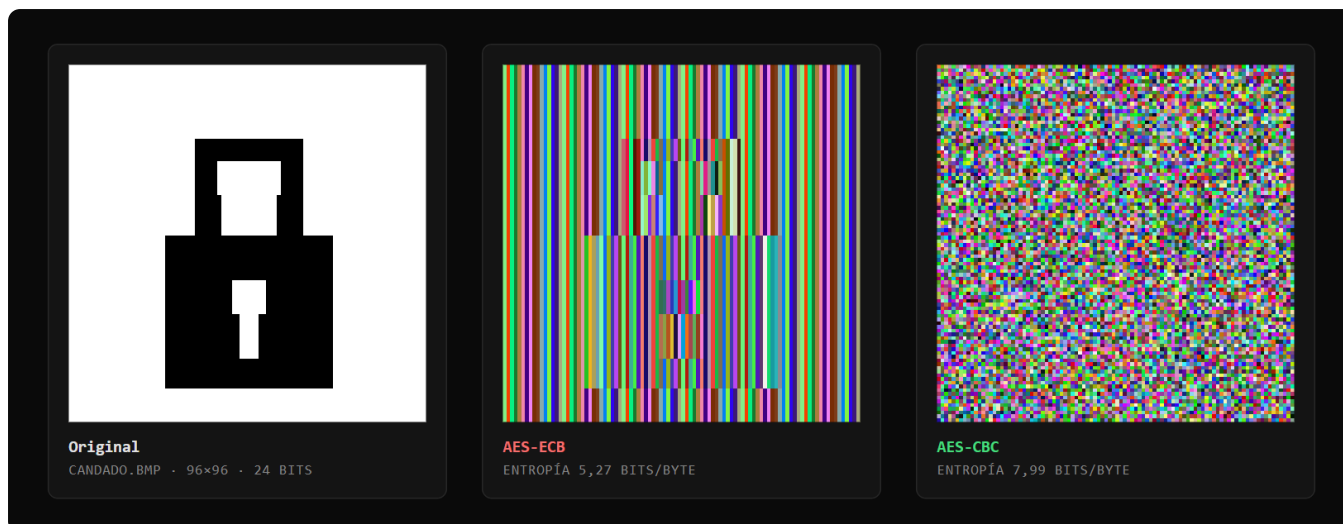
Qué hace cada bloque, con sus parámetros:



Se separan los 54 bytes de cabecera del BMP (que deben quedar intactos para que siga siendo una imagen válida) de los datos de píxel. Solo se cifran los píxeles; luego se vuelve a pegar la cabecera y se renderiza. La imagen es de 96×96 a 24 bits: 27 648 bytes de píxeles, múltiplo exacto de 16, por eso basta con `NoPadding`.

Con ECB se reconoce el candado: la imagen tiene grandes zonas de un solo color, es decir, muchísimos bloques de 16 bytes idénticos. ECB los convierte a todos en el mismo bloque cifrado, de modo que las regiones uniformes siguen siendo uniformes y el contorno se dibuja solo. De los 1 728 bloques de la imagen, solo 14 son distintos.

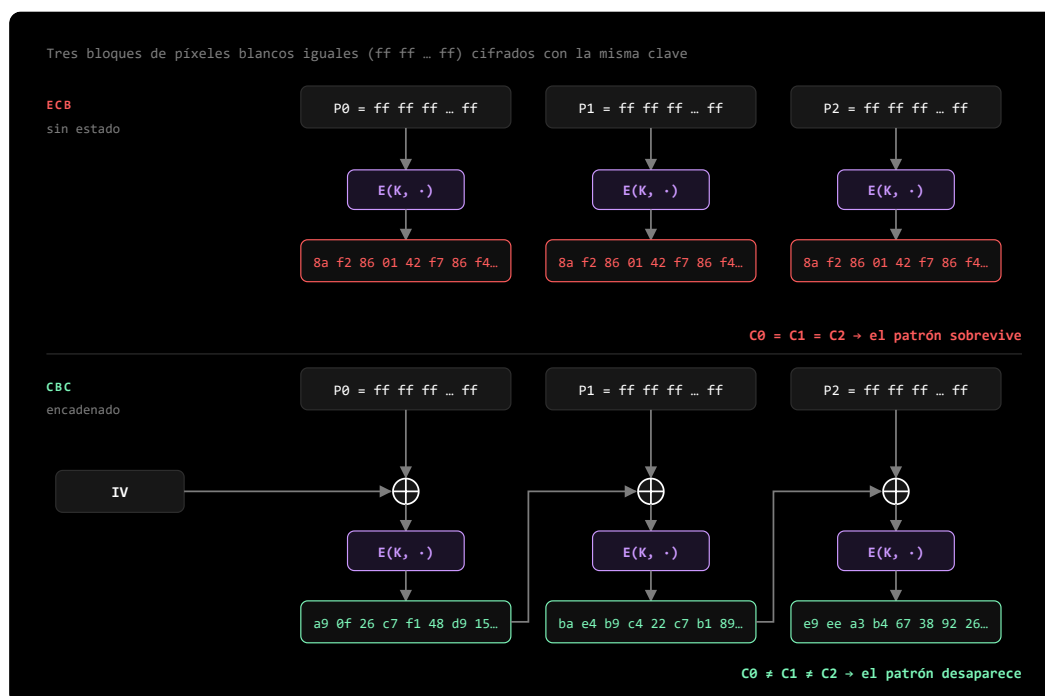
El paso del reto: abre el bloque *AES Encrypt* (botón ↗) y cambia **Mode** de `ECB/NoPadding` a `CBC/NoPadding`. Vuelve a abrir *Render Image* y mira la pestaña **Vista de CyberChef** de la salida: ahora es ruido uniforme, sin figura.



Las dos imágenes cifradas son la salida real de «Render Image» en CipherFlow, ampliadas sin suavizado. Las franjas verticales de ECB aparecen porque un bloque de 16 bytes no coincide con un número entero de píxeles de 3 bytes; aun así, el candado se distingue perfectamente.

Por qué

- ECB: $C_i = E(K, P_i)$. No hay estado entre bloques. $P_i = P_j \Rightarrow C_i = C_j$.
- CBC: $C_i = E(K, P_i \oplus C_{i-1})$ (con $C_{-1} = IV$). Cada bloque se mezcla (XOR) con el cifrado anterior antes de cifrarse, así que dos bloques de entrada iguales producen cifrados distintos y el patrón se rompe.



Los bytes cifrados del diagrama son los primeros de la salida real de AES Encrypt en el lab: un bloque de píxeles blancos ($ff...ff$) siempre da $8a\ f2\ 86\ 01...$ en ECB, mientras que en CBC cada bloque sale distinto.

En CipherFlow puedes comprobarlo en el propio bloque *AES Encrypt*. En **Salida**, con ECB el volcado hexadecimal repite la misma línea $8a\ f2\ 86\ 01\ 42\ f7\ 86\ f4\ 09\ 30\ 7c\ 1a\ 3f\ 7e\ aa\ ac$ una y otra vez; en **Proceso**, la entropía de salida se queda en **5,27 bits/byte**, lejos de los 8 de unos datos aleatorios:



```
python3 -c "import json,base64;s=open('src/labs.data.ts',encoding='utf8').read();d=json.loads(s[s.index('{'):s.rindex('}')+1]);open('candado.bmp','wb').write(base64.b64decode(d['ecb']['bmpB64']))"
```

Separa cabecera y píxeles, como *Take bytes* y *Drop bytes*:

```
head -c 54 candado.bmp > cabecera.bin      # 54 B
tail -c +55 candado.bmp > pixeles.bin      # 27 648 B
```

Cifra los píxeles en los dos modos (misma clave que el lab; en CBC el lab usa la clave también como IV) y vuelve a pegar la cabecera, como *Unir flujos*:

```
K=2b7e151628aed2a6abf7158809cf4f3c
openssl enc -aes-128-ecb -K $K -nopad -in pixeles.bin -out pix-ecb.bin
openssl enc -aes-128-cbc -K $K -iv $K -nopad -in pixeles.bin -out pix-cbc.bin
cat cabecera.bin pix-ecb.bin > candado-ecb.bmp
cat cabecera.bin pix-cbc.bin > candado-cbc.bmp
```

Abre `candado-ecb.bmp` y `candado-cbc.bmp` con cualquier visor de imágenes: verás lo mismo que en *Render Image*. Los primeros bytes cifrados son los mismos que muestra *AES Encrypt*:

```
xxd -s 54 -l 32 candado-ecb.bmp
# 00000036: 8af2 8601 42f7 86f4 0930 7c1a 3f7e aaac ← el mismo bloque...
# 00000046: 8af2 8601 42f7 86f4 0930 7c1a 3f7e aaac ← ...repetido
xxd -s 54 -l 32 candado-cbc.bmp
# 00000036: a90f 26c7 f148 d915 5867 b8d6 6d58 601e
# 00000046: bae4 b9c4 22c7 b189 ab88 6f4a 4609 9f61
```

Cuenta cuántos bloques de 16 bytes distintos hay (una línea de `xxd -p -c 16` = un bloque):

```
xxd -p -c 16 pix-ecb.bin | sort -u | wc -l      # 14 (de 1728)
xxd -p -c 16 pix-cbc.bin | sort -u | wc -l      # 1728 (todos distintos)
xxd -p -c 16 pix-ecb.bin | sort | uniq -c | sort -rn | head -3
# 1202 8af2860142f786f409307c1a3f7eaaac ← fondo blanco
# 310 7df76b0c1ab899b33e42f047b91b546f ← cuerpo negro del candado
# 41 96de26c62620b7f35da2aeb7a269a7fb
```

Y la entropía, la misma medida que el panel **Proceso**, con un pequeño script `entropia.py`:

```
import math, sys
from collections import Counter
d = open(sys.argv[1], 'rb').read()
print(f"{-sum(c/len(d)*math.log2(c/len(d)) for c in Counter(d).values()):.2f} bits/byte")
```

```
python3 entropia.py pixeles.bin      # 0.78 bits/byte
python3 entropia.py pix-ecb.bin      # 5.27 bits/byte
python3 entropia.py pix-cbc.bin      # 7.99 bits/byte
```

Impacto real

ECB no solo afecta imágenes: filtra cualquier estructura repetida (campos fijos de un formato, cabeceras, valores por defecto). Por eso está desaconsejado para prácticamente todo. Un atacante que observa tráfico cifrado con ECB puede deducir cuándo se repite un mensaje o localizar campos conocidos.

Mitigación

Nunca uses ECB para datos con estructura. Usa un modo autenticado (**AES-GCM**, ChaCha20-Poly1305) que además de ocultar patrones detecta manipulaciones. Si el escenario obliga a CBC, combínalo con un MAC (encrypt-then-MAC) y un IV aleatorio por mensaje. (En este lab el IV es igual a la clave solo para simplificar; en un sistema real eso es otro error.)

Lab 2 — Nonce reutilizado (two-time pad)

Vulnerabilidad: un cifrado de flujo (RC4, AES-CTR, ChaCha20, o el one-time pad) produce un flujo de clave k que se combina con el mensaje mediante XOR: $c = m \oplus k$. La seguridad depende de que k **nunca se repita**. Si dos mensajes se cifran con el mismo k (mismo nonce/IV, o la misma semilla), la clave se puede cancelar.

El reto

Tienes $c1$ y $c2$ (64 bytes cada uno), cifrados con el mismo keystream. Recupera $m1$ y $m2$ sin la clave. Sabes que $m1$ empieza por «Nos vemos a».

■ LABORATORIO · INTERMEDIO

2 · Nonce reutilizado (two-time pad)

REUTILIZAR EL FLUJO DE CLAVE EN UN CIFRADO DE FLUJO

🎯 OBJETIVO

Recuperar dos mensajes cifrados con el MISMO flujo de clave, sin conocer la clave.

EL RETO

Interceptaste dos mensajes ($c1$ y $c2$) cifrados con el mismo keystream: $c = m \oplus k$. El bloque «XOR de dos flujos» ya calcula $c1 \oplus c2$, que es igual a $m1 \oplus m2$ (la clave se cancela). Sabemos que $m1$ empieza por «Nos vemos a». Úsalo como cuña (crib dragging) para destapar el arranque de $m2$, y desde ahí reconstruye ambos mensajes.

🔍 PISTAS (4) ▾

LO LOGRASTE CUANDO...

Se leen los dos mensajes completos en español.

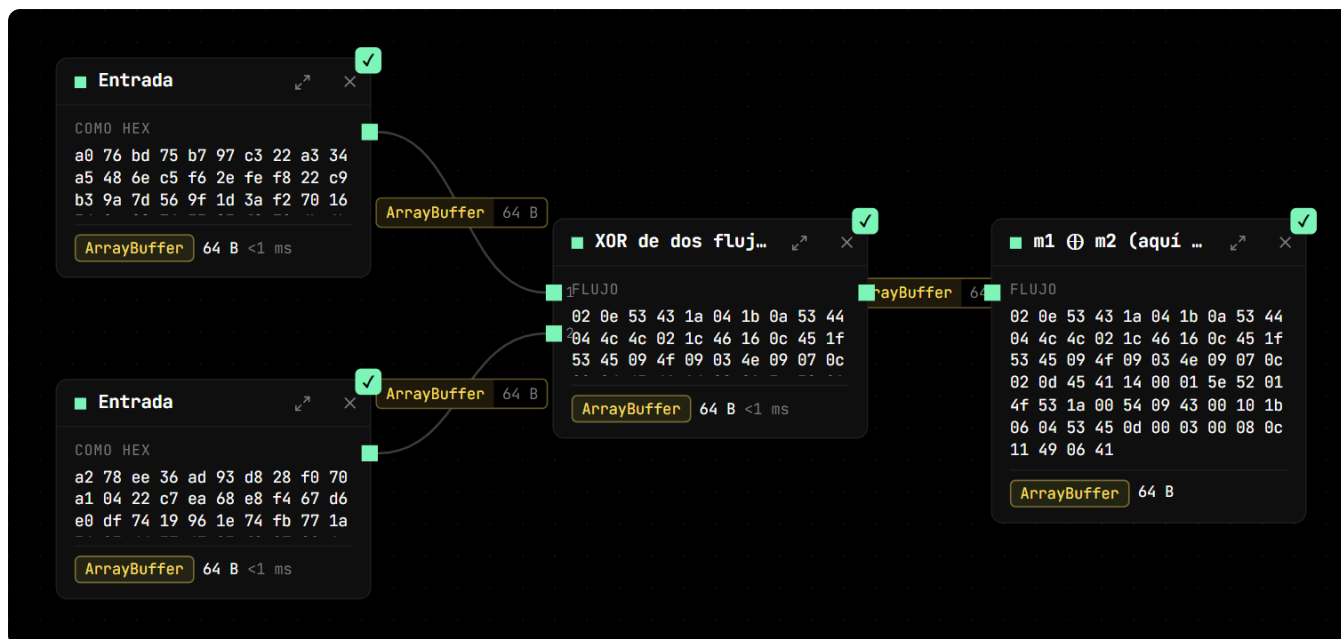
📄 VER SOLUCIÓN (WRITEUP)

AL LIENZO →

Solución

El bloque *XOR de dos flujos* ya calcula:

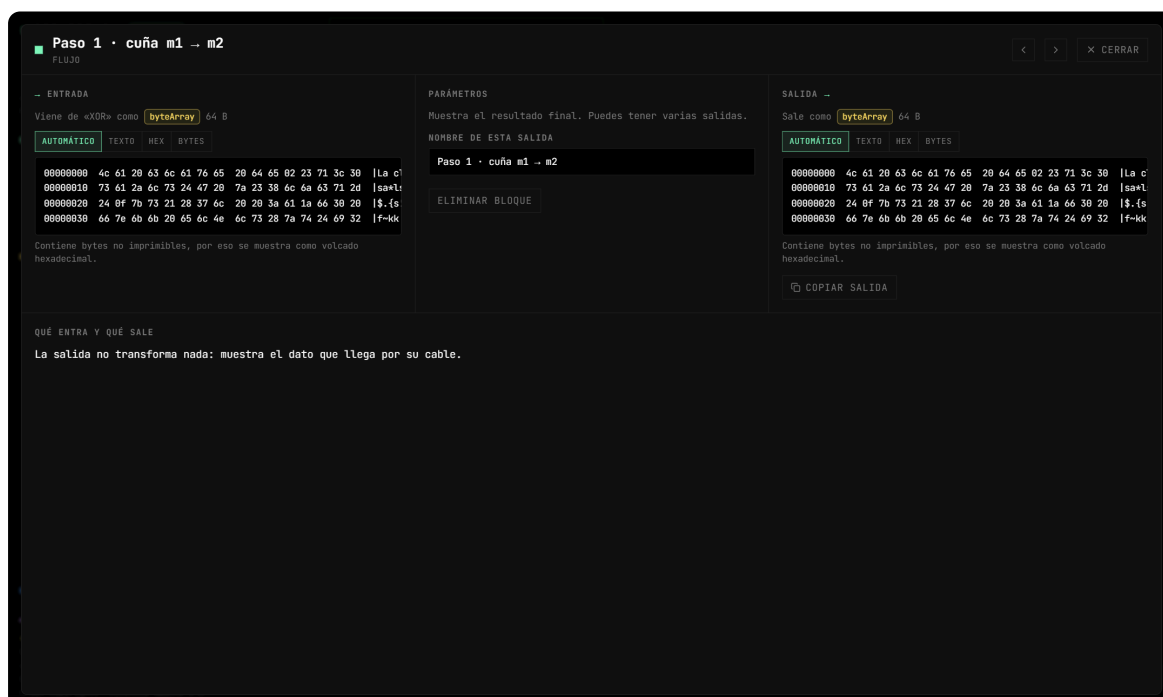
$$c1 \oplus c2 = (m1 \oplus k) \oplus (m2 \oplus k) = m1 \oplus m2$$



El keystream desaparece porque $k \oplus k = 0$. Queda $m1 \oplus m2$, que **no** es legible, pero contiene toda la información: si conoces un trozo de uno de los mensajes, obtienes el mismo trozo del otro.

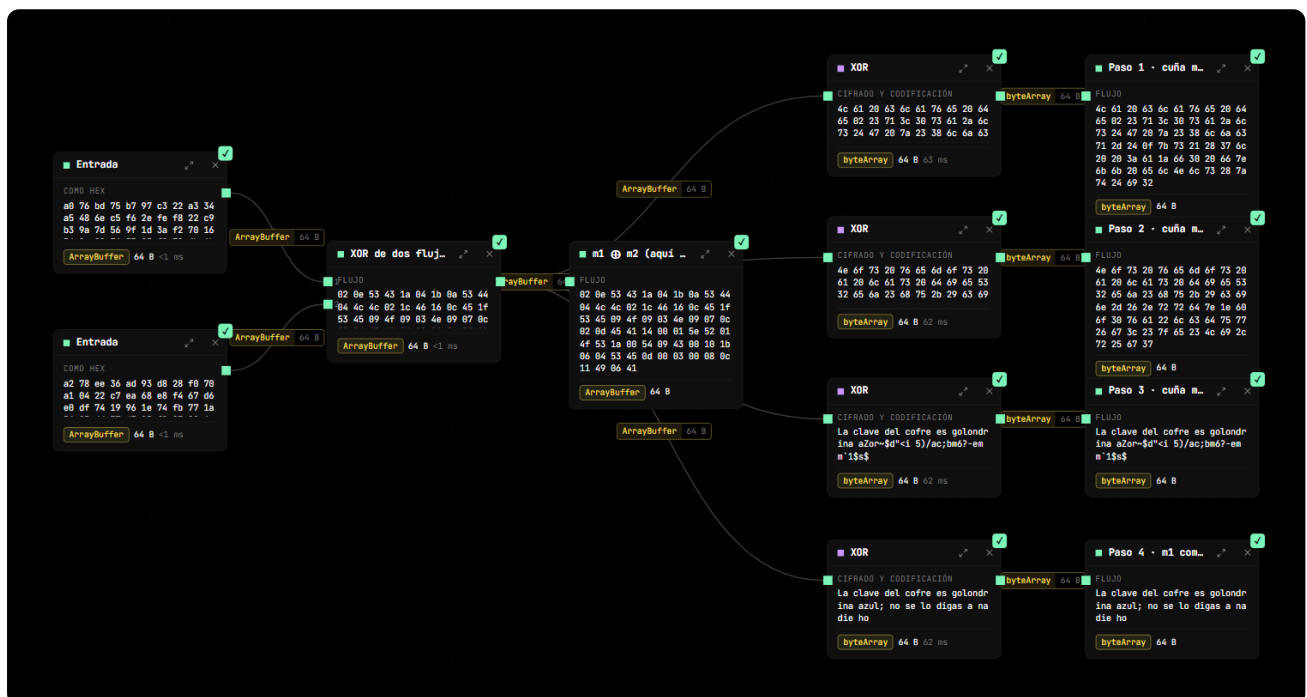
Crib dragging paso a paso

1. Añade un bloque **XOR** conectado a la salida del *XOR de dos flujos*. En su parámetro **Key** elige el formato **UTF8** y escribe la contraseña conocida: `Nos vemos a`. El bloque XOR repite la clave a lo largo de todo el dato, así que en los primeros 11 bytes calcula $(m1 \oplus m2) \oplus \text{"Nos vemos a"}$. Como en esa posición $m1 = \text{"Nos vemos a"}$, el resultado es **el arranque de m2**: `La clave de`. A partir del byte 11 la clave se repite sobre bytes que no le corresponden y sale basura.



- Amplía la cuña adivinando.** La clave de... pide a gritos La clave del , y una clave suele ser de algo: prueba La clave del cofre como nueva clave del XOR. Ahora la cuña es un trozo de m2, así que lo que sale es m1: Nos vemos a las die . Si la suposición fuera incorrecta, el lado de m1 saldría ilegible y sabrías que hay que corregirla.
- Alterna.** «las die» se completa como las diez en el muelle ; úsalo como cuña de m1 y sale La clave del cofre es golondrina a... . Cada trozo revelado de un mensaje destapa el mismo trozo del otro, y el español ayuda: las palabras se completan solas.
- Con m1 completo como clave, sale m2 completo.

Puedes tener todos los intentos a la vez en el lienzo, un bloque XOR por cuña colgando del mismo *XOR de dos flujos*:



$c1 \oplus c2 = m1 \oplus m2$ — cada cuña conocida de un mensaje destapa el mismo tramo del otro

Salida real del bloque XOR (clave UTF8 = la cuña, que se repite): lo legible es exactamente la longitud de la cuña.

CUÑA (CLAVE DEL XOR)

RESULTADO: $(M1 \oplus M2) \oplus \text{CUÑA}$

Paso 1 cuña de m1 → sale m2

"Nos vemos a"

La clave de~#q<0sa*1s\$G z

Paso 2 cuña de m2 → sale m1

"La clave del cofre "

Nos vemos a las dieS2ej#hu+)cin-&

Paso 3 cuña de m1 → sale m2

"Nos vemos a las diez en el muelle "

La clave del cofre es golondrina aZor~\$d<i 5)/a

Paso 4 cuña de m1 → sale m2

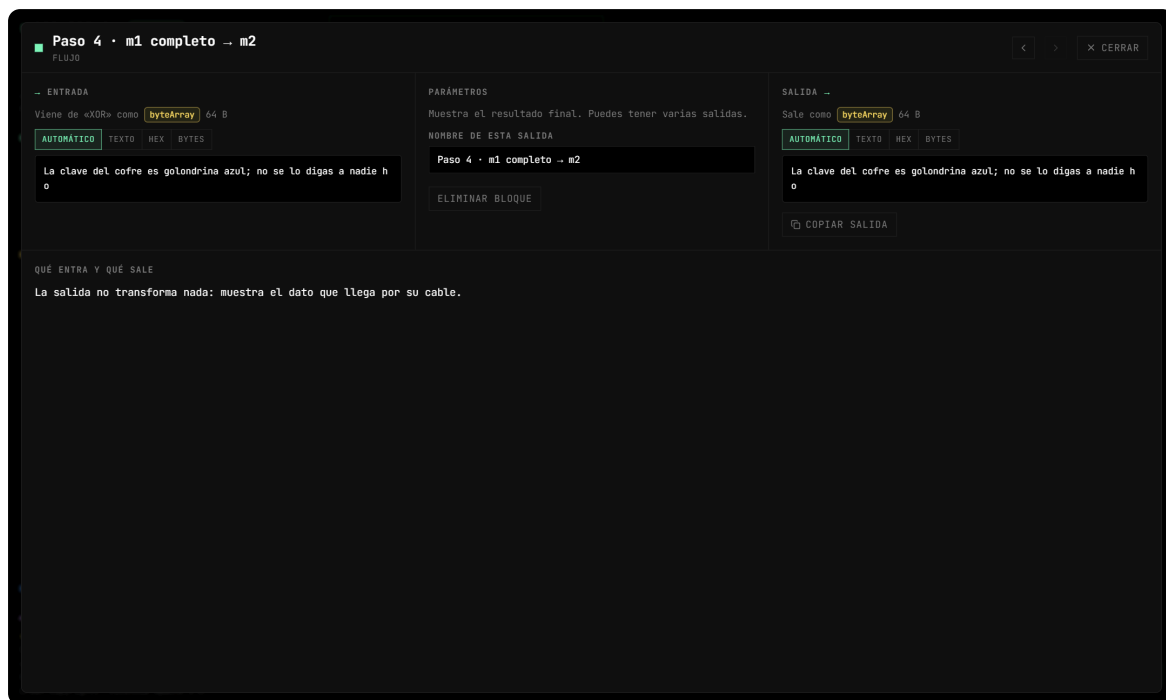
"Nos vemos a las diez en el muelle numero..."

m1 completo → m2 completo (64 B)
La clave del cofre es golondrina azul; no se lo digas a nadie ho

Verde: texto recuperado · Rojo: la cuña se repite sobre bytes que aún no conoces y sale basura.

Resultado:

- `m1 = "Nos vemos a las diez en el muelle numero siete, trae el maletin."`
- `m2 = "La clave del cofre es golondrina azul; no se lo digas a nadie ho"`



Ojo con la longitud. Los dos cifrados tienen 64 bytes, así que solo se recuperan 64 caracteres de cada mensaje. El `m2` original terminaba en «...a nadie hoy.», pero esos dos últimos caracteres nunca llegaron al cifrado del lab: no hay keystream ni `c2` para ellos.

Atajo con texto claro conocido

Si conocieras `m1` completo, el flujo de clave se recupera directamente:

$$\begin{aligned} k &= c1 \oplus m1 && \text{(bloque XOR de dos flujos)} \\ m2 &= c2 \oplus k && \text{(otro XOR de dos flujos)} \end{aligned}$$

y descifras cualquier otro mensaje cifrado con ese mismo `k` (hasta la longitud de `k` que hayas recuperado).

Verifícalo con otras herramientas

`openssl` no trae un XOR de dos archivos, así que aquí basta con Python (ver [requisitos](#)). Guarda esto como `crib.py`; hace lo mismo que *XOR de dos flujos* seguido de un bloque *XOR* con la cuña como clave:

```
import sys
c1 = bytes.fromhex('a076bd75b797c322a334a5486ec5f62efef822c9b39a7d569f1d3af27016349e98765385f87
9db6b111a71cf85f1cb42e7d553e291a379a8cd86905ccf67438a')
c2 = bytes.fromhex('a278ee36ad93d828f070a10422c7ea68e8f467d6e0df7419961e74fb771a3693dd374785f92
7896a5e496bcfd1f88842f7ce55e6c2e674a8ce869850de2e45cb')
x = bytes(a ^ b for a, b in zip(c1, c2)) # c1 ⊕ c2 = m1 ⊕ m2
cuna = sys.argv[1].encode()
res = bytes(b ^ cuna[i % len(cuna)] for i, b in enumerate(x)) # la cuña se repite, como en e
l bloque XOR
print(''.join(chr(c) if 32 ≤ c < 127 else '.' for c in res)) # '.' = byte no imprimible
```

Y repite los pasos del crib dragging:

```
python3 crib.py "Nos vemos a"
# La clave de.#q<0sa*ls$G z#8ljcq-$.{s!(7l :a.f0 f~kk eNls(zt$i2
python3 crib.py "La clave del cofre "
# Nos vemos a las dieS2ej#hu+)cin-&.rrd~.`o0va"lcduw&g<#.e#Li,r%g7
python3 crib.py "Nos vemos a las diez en el muelle "
# La clave del cofre es golondrina aZor~$d"<i 5)/ac;bm6?-em m`1$s$
python3 crib.py "Nos vemos a las diez en el muelle numero siete, trae el maletin."
# La clave del cofre es golondrina azul; no se lo digas a nadie ho
```

Prueba también una cuña equivocada, por ejemplo `python3 crib.py "Hola amigo"`: el principio sale ilegible y sabes que esa suposición no encaja.

Por qué

XOR es su propia inversa y es asociativo/conmutativo. Toda la seguridad del one-time pad vive en que `k` sea aleatorio y de un solo uso. Reutilizarlo una vez lo degrada a un cifrado de sustitución sobre `m1 ⊕ m2`, que se rompe con estadística y cuñas.

Impacto real

Es un patrón recurrente: el **WEP** de Wi-Fi reutilizaba IVs de 24 bits de RC4; **Microsoft Office** y **PPTP** tuvieron fallos de nonce; y muchos usos incorrectos de **AES-CTR** o **GCM** repiten el nonce. Repetir el nonce en GCM es aún peor: además de exponer los textos, permite recuperar la clave de autenticación.

Mitigación

Nonce único por mensaje, siempre. Genera el nonce de forma aleatoria (96 bits en GCM) o con un contador que nunca se reinicie con la misma clave. Rota la clave antes de agotar el espacio de nonces.

Lab 3 — Colisión de MD5

Vulnerabilidad: una función hash criptográfica debe ser *resistente a colisiones*: debe ser inviable encontrar dos entradas distintas con el mismo hash. MD5 se rompió en 2004 (Wang et al.) y hoy se generan colisiones en segundos. SHA-1 cayó en 2017 (ataque *SHAttered* de Google). Un hash sin resistencia a colisiones no sirve para integridad ni para firmas.

El reto

Los bloques A y B son dos archivos distintos (difieren en 6 bytes). Demuestra que MD5 no distingue archivos y explica qué ataque habilita.

LABORATORIO · BÁSICO

3 · Colisión de MD5

MD5 (Y SHA-1) YA NO RESISTEN COLISIONES

🎯 OBJETIVO

Demstrar que un hash roto no sirve para garantizar integridad.

EL RETO

Los bloques A y B son dos archivos DISTINTOS (difieren en 6 bytes). El flujo ya calcula el MD5 de cada uno: son idénticos. Ahora agrega a cada rama un bloque «SHA2» (256) y compara. Responde: si un antivirus o una tienda de apps identifica los archivos por su MD5, ¿qué ataque permite esto?

💡 PISTAS (3) ▾

LO LOGRASTE CUANDO...

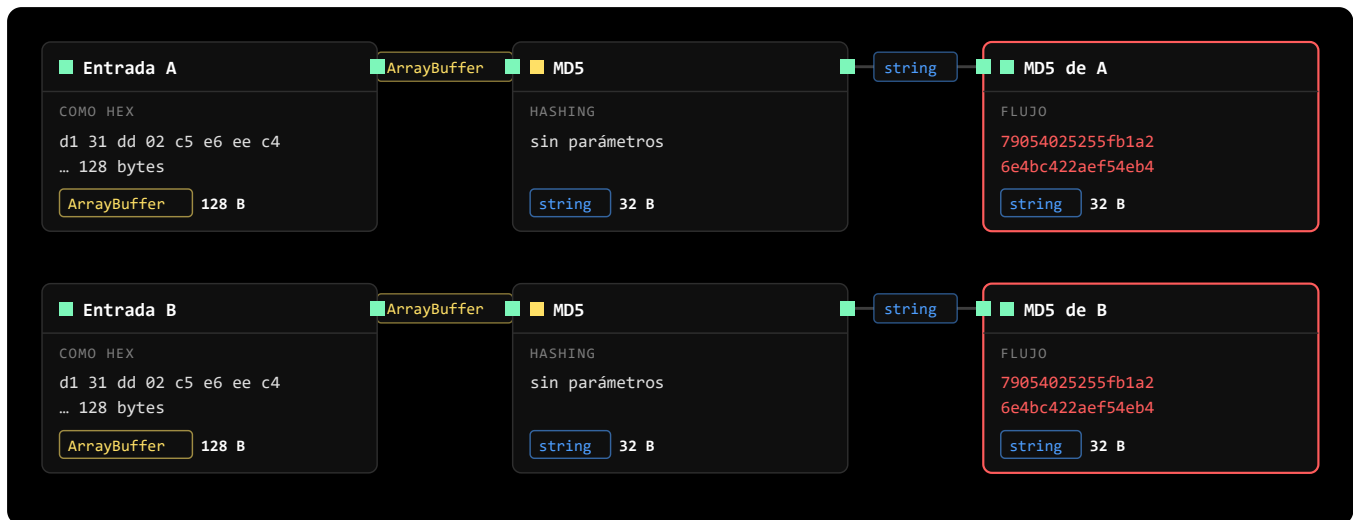
Los dos MD5 coinciden y los dos SHA-256 difieren.

🔗 VER SOLUCIÓN (WRITEUP)

AL LIENZO →

Solución

El flujo calcula `MD5(A)` y `MD5(B)`:



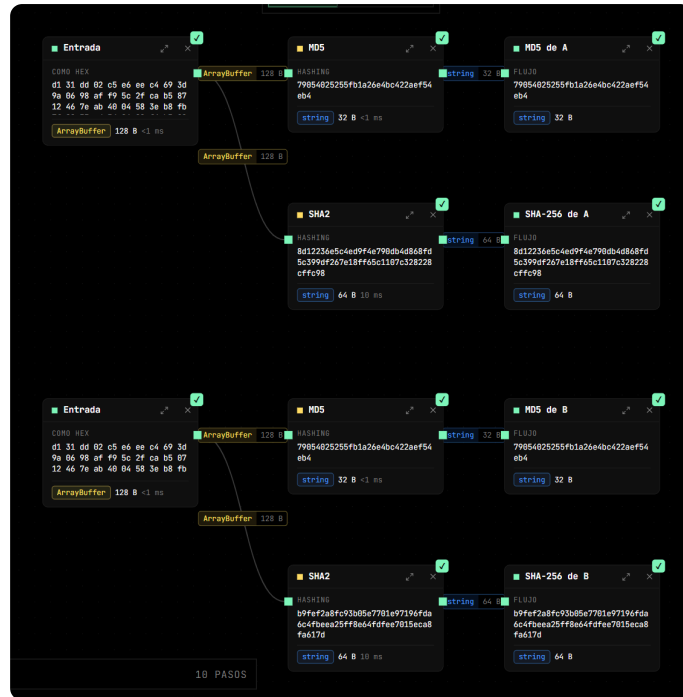
Idénticos, aunque $A \neq B$. Este es el par clásico de Wang et al.: dos bloques que difieren en las posiciones 19, 45, 59, 83, 109 y 123. Puedes verlo abriendo cada *Entrada* y comparando su volcado hexadecimal:

Bloque A y bloque B: 128 bytes cada uno, distintos solo en 6 posiciones
 Posiciones 19, 45, 59, 83, 109 y 123 (resaltadas). Todo lo demás es idéntico.

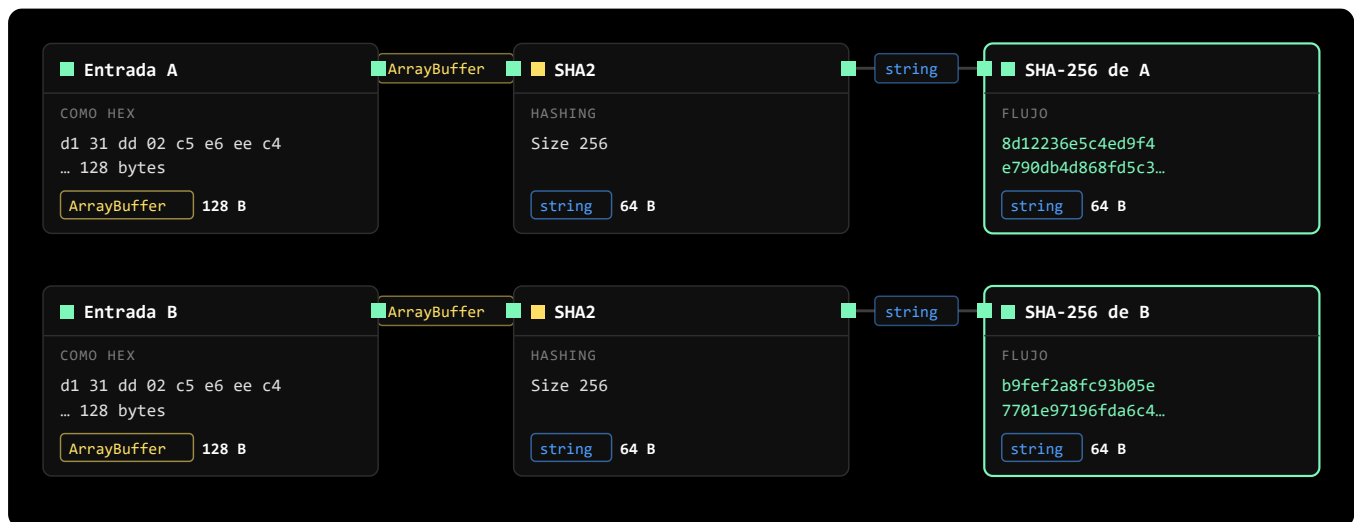
A	B
00 d1 31 dd 02 c5 e6 ee c4 69 3d 9a 06 98 af f9 5c	00 d1 31 dd 02 c5 e6 ee c4 69 3d 9a 06 98 af f9 5c
10 2f ca b5 87 12 46 7e ab 40 04 58 3e b8 fb 7f 89	10 2f ca b5 07 12 46 7e ab 40 04 58 3e b8 fb 7f 89
20 55 ad 34 06 09 f4 b3 02 83 e4 88 83 25 71 41 5a	20 55 ad 34 06 09 f4 b3 02 83 e4 88 83 25 f1 41 5a
30 08 51 25 e8 f7 cd c9 9f d9 1d bd f2 80 37 3c 5b	30 08 51 25 e8 f7 cd c9 9f d9 1d bd 72 80 37 3c 5b
40 d8 82 3e 31 56 34 8f 5b ae 6d ac d4 36 c9 19 c6	40 d8 82 3e 31 56 34 8f 5b ae 6d ac d4 36 c9 19 c6
50 dd 53 e2 b4 87 da 03 fd 02 39 63 06 d2 48 cd a0	50 dd 53 e2 34 87 da 03 fd 02 39 63 06 d2 48 cd a0
60 e9 9f 33 42 0f 57 7e e8 ce 54 b6 70 80 a8 0d 1e	60 e9 9f 33 42 0f 57 7e e8 ce 54 b6 70 80 28 0d 1e
70 c6 98 21 bc b6 a8 83 93 96 f9 65 2b 6f f7 2a 70	70 c6 98 21 bc b6 a8 83 93 96 f9 65 ab 6f f7 2a 70

MD5	A: 79054025255fb1a26e4bc422aef54eb4	B: 79054025255fb1a26e4bc422aef54eb4	iguales
SHA-256	A: 8d12236e5c4ed9f4e790db4d...	B: b9fef2a8fc93b05e7701e971...	distintos

El paso del reto: agrega a cada rama un bloque **SHA2** (Size 256, el valor por defecto) conectado a su *Entrada*, y una *Salida* para verlo. El lienzo queda así:



Aislando solo las ramas nuevas:



SHA-256 sí los distingue: dos hashes completamente diferentes. La conclusión es directa: si un sistema identifica archivos por su MD5, dos archivos distintos le parecen el mismo.

Por qué importan las 6 diferencias

MD5 procesa el mensaje en bloques de 64 bytes; A y B ocupan dos bloques cada uno. La colisión no es casual: los seis bytes que cambian están calculados para que las diferencias se cancelen dentro de la función de compresión de MD5 (rutas diferenciales): el primer bloque introduce una diferencia controlada en el estado interno y el segundo la anula. Al terminar, el estado es idéntico y el hash también. Como MD5 encadena bloques, además se puede añadir cualquier sufijo común a A y B y seguirán colisionando.

El ataque de Wang produce pares como este; el de **prefijo elegido** (*chosen-prefix collision*, Stevens 2007) va más allá: parte de dos prefijos arbitrarios distintos y calcula bloques que hacen colisionar ambos. Eso es lo que permitió falsificar un certificado de CA real.

Verifícalo con otras herramientas

Crea los dos archivos a partir del hex de las *Entradas* (ver [requisitos](#)):

```
echo d131dd02c5e6eec4693d9a0698aff95c2fcbab58712467eab4004583eb8fb7f8955ad340609f4b30283e4888325
71415a085125e8f7cdc99fd91dbdf280373c5bd8823e3156348f5bae6dacd436c919c6dd53e2b487da03fd02396306d
248cda0e99f33420f577ee8ce54b67080a80d1ec69821bcb6a8839396f9652b6ff72a70 | xxd -r -p > a.bin
echo d131dd02c5e6eec4693d9a0698aff95c2fcbab50712467eab4004583eb8fb7f8955ad340609f4b30283e4888325
f1415a085125e8f7cdc99fd91dbdf280373c5bd8823e3156348f5bae6dacd436c919c6dd53e23487da03fd02396306d
248cda0e99f33420f577ee8ce54b67080280d1ec69821bcb6a8839396f965ab6ff72a70 | xxd -r -p > b.bin
```

Compáralos byte a byte:

```
cmp -l a.bin b.bin
# 20 207 7
# 46 161 361
# 60 362 162
# 84 264 64
# 110 250 50
# 124 53 253
```

`cmp -l` cuenta las posiciones desde 1 y muestra los valores en **octal**: `20 207 7` es la posición 19 (contando desde 0, como el volcado de CipherFlow), con `0x87` en A y `0x07` en B. Son las 6 diferencias del diagrama.

Los hashes, con `coreutils` o con `openssl dgst`:

```
md5sum a.bin b.bin
# 79054025255fb1a26e4bc422aef54eb4 a.bin
# 79054025255fb1a26e4bc422aef54eb4 b.bin ← iguales
sha256sum a.bin b.bin
# 8d12236e5c4ed9f4e790db4d868fd5c399df267e18ff65c1107c328228cffc98 a.bin
# b9fef2a8fc93b05e7701e97196fda6c4fbee25ff8e64fdfee7015eca8fa617d b.bin
openssl dgst -sha1 a.bin b.bin
# SHA1(a.bin)= a34473cf767c6108a5751a20971f1fd6ba97690a
# SHA1(b.bin)= 4283dd2d70af1ad3c2d5fdc917330bf502035658
```

SHA-1 también los distingue: esta colisión está fabricada específicamente contra MD5. Las colisiones de SHA-1 existen (SHAttered), pero hay que construirlas contra SHA-1.

Para ver el ataque de sufijo común en acción, añade los mismos bytes al final de ambos archivos y vuelve a calcular: los MD5 siguen coincidiendo.

```
printf 'cualquier sufijo' | tee -a a.bin >> b.bin
md5sum a.bin b.bin # de nuevo iguales (aunque distintos de 7905...)
```

Impacto real

- **Certificados X.509 falsos:** en 2008 se creó un certificado de CA fraudulento con el mismo MD5 que uno legítimo.
- **Flame (2012):** malware que falsificó una firma de Microsoft usando una colisión de MD5 para aparentar ser una actualización de Windows.
- **Integridad rota:** un `md5sum` que coincide ya no garantiza que un archivo no fue sustituido.

Mitigación

No uses MD5 ni SHA-1 para nada que dependa de resistencia a colisiones (firmas, certificados, verificación de descargas, deduplicación con implicaciones de seguridad). Usa **SHA-256** o superior. Para autenticar mensajes con clave, usa **HMAC** (ver el Lab 5 y el ejemplo «SHA-256 y HMAC»; el bloque *SHA2* de CipherFlow tiene además la explicación paso a paso en su panel **Proceso**). MD5 solo sobrevive como suma de verificación no adversarial (detectar corrupción accidental).

Lab 4 — Análisis de frecuencias

Vulnerabilidad: un cifrado de sustitución monoalfabética reemplaza cada letra por otra fija. Hay $26! \approx 4 \times 10^{26}$ claves posibles, así que la fuerza bruta es inviable... pero la sustitución **no oculta la estadística del idioma**. En español la E y la A son con diferencia las más frecuentes; esa huella pasa intacta al texto cifrado y delata la clave.

El reto

Descifra el texto sin la clave, usando solo las frecuencias del español. El bloque *Frequency distribution* te da el conteo; el bloque *Substitute* aplica tu mapa.

■ LABORATORIO · INTERMEDIO

4 · Análisis de frecuencias

LA SUSTITUCIÓN MONOALFABÉTICA CONSERVA LA HUELLA DEL IDIOMA

🎯 OBJETIVO

Descifrar un texto sin la clave, usando solo la estadística del español.

EL RETO

El texto está cifrado con una sustitución monoalfabética (cada letra se cambió por otra fija). El bloque «Frequency distribution» muestra qué símbolos son más comunes. Usa el bloque «Substitute» para ir armando el alfabeto: en «Ciphertext» pon las letras cifradas y en «Plaintext» tus hipótesis. Recupera el mensaje.

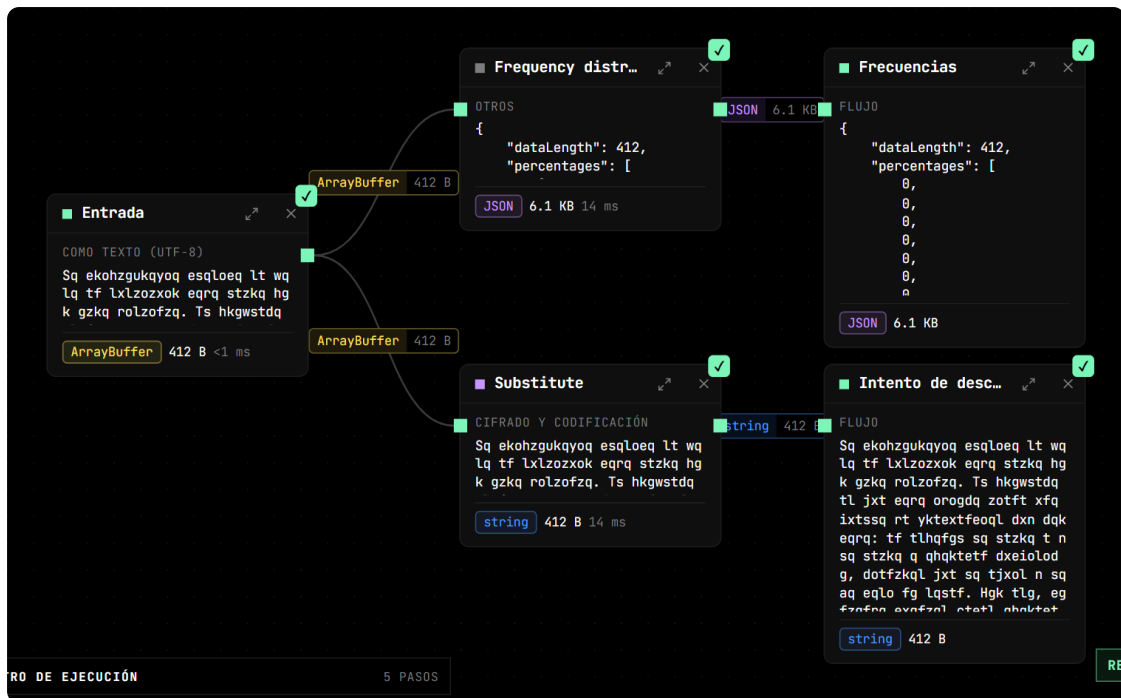
🔍 PISTAS (4) ▾

LO LOGRASTE CUANDO...

El texto descifrado es español legible.

🔗 VER SOLUCIÓN (WRITEUP)

AL LIENZO →



Solución

1. **Mira las frecuencias.** Abre el bloque *Frequency distribution* y, en **Salida**, la pestaña **Vista de CyberChef**. La tabla está ordenada por valor de byte (en hexadecimal: `71` = `q`, `74` = `t` ...); marca **Show ASCII** en sus parámetros si prefieres ver el carácter. El espacio (`20`) domina con un 17 %; entre las letras, busca las barras más largas.

Frequency distribution

OTROS, ENTRA ARRAYBUFFER Y SALE HTML

Viene de «Entrada» como **ArrayBuffer** 412 B

AUTOMÁTICO TEXTO HEX BYTES

```
Sq ekohzgukqyoq esqloeq lt wq tf lxlzoxok eqrq stzkq hgk gzk q rolzofzq. Ts hkgwstdq tl jxt eqrq orogdq zotft xfq itssq rt yktextfeqql ddn dqkeqqr: tf tlhqfqs sq stzkq t n sq stzkq q qhq ktetf dxeiolodg, dotfzkl jxt sq tjxol n sq aq eqlo fg lqstf. H gk tlg, egfzqfng exqfzql ctetl qhqtet eqrq lodwsg rts dtflapt eoykqng n egdhqkqfng egf tlg itssq, lt hxtet qrocofak ts qsyqw tgz ltektzq lof egfgetk sq esqet.
```

PARÁMETROS

☐ Show 0%

☐ Show ASCII

► Descripción original de CyberChef (Inglés)

ELIMINAR BLOQUE

SALIDA ...

Salte como **JSON** 6.1 KB, calculado en 13.8 ms

AUTOMÁTICO TEXTO HEX BYTES **VISTA DE CYBERCHEF**

Total data length: 412 Number of bytes represented: 30 Number of bytes not represented: 226

Byte	Percentage
20	17.23%
2c	0.73%
2e	0.73%
3a	0.24%
48	0.24%
53	0.24%
54	0.24%

COPIAR SALIDA VOLVER A EJECUTAR

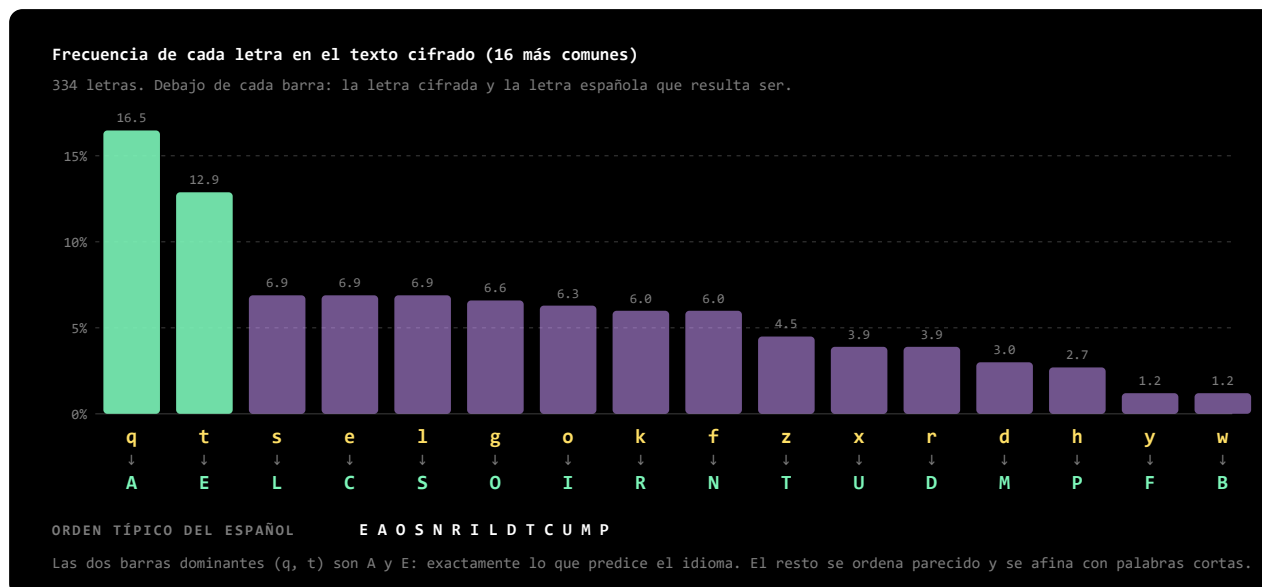
QUÉ ENTRA Y QUÉ SALE

1. Llega un dato de tipo **ArrayBuffer** con 412 B.
2. La operación **Frequency distribution** se ejecuta con los parámetros elegidos.
3. Sale un dato de tipo **JSON** con 6.1 KB. CyberChef además genera una vista enriquecida («Vista de CyberChef» en Salida).

Tamaño entrada	412 B
Tamaño salida	6.1 KB
Entropía entrada	4.05 bits/byte
Entropía salida	1.98 bits/byte

La entropía mide qué tan impredecibles son los bytes: un texto normal ronda 4

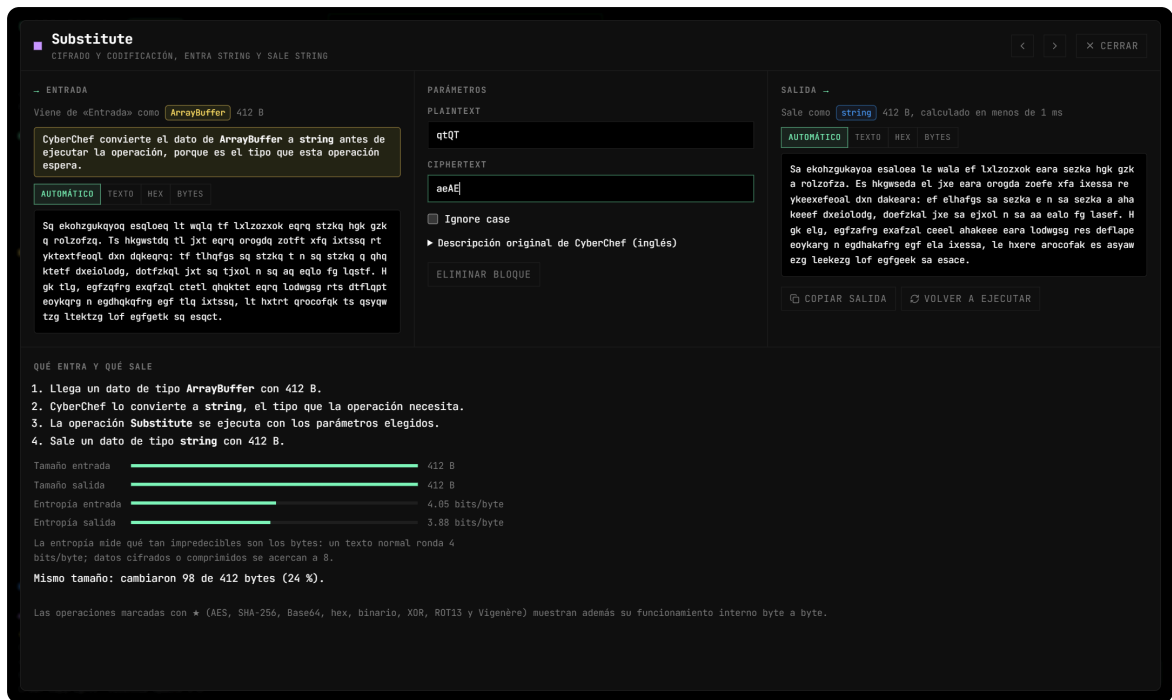
Ordenadas de mayor a menor, las letras del cifrado quedan así:



- Empareja por frecuencia.** Orden aproximado en español: E A O S N R I L D T C U M P. Dos letras destacan claramente: q (16,5 %) y t (12,9 %). Son A y E. A partir de ahí las frecuencias se parecen mucho entre sí (s, e, l empatan al 6,9 %), así que el orden ya no basta: toca usar la estructura.
- Usa la estructura.** Las palabras de una letra son A, E, O o Y. Las de dos muy repetidas: DE, LA, EL, EN, SE. Los dígrafos frecuentes (QU, CH, RR) y las terminaciones (-CIÓN, -MENTE, -ADO) confirman letras. Por ejemplo, el texto contiene sq y t sueltas: si q = A y t = E, sq es «LA», y s = L.
- Aplica el mapa con Substitute.** Este bloque cambia cada carácter de Plaintext por el carácter de la misma posición en Ciphertext. Para descifrar, por tanto:
 - en Plaintext van las letras cifradas;
 - en Ciphertext van tus hipótesis (las letras en claro).

Los nombres de los campos engañan: piensa en ellos como «de» y «a». La opción *Ignore case* está desactivada, así que incluye minúsculas y mayúsculas.

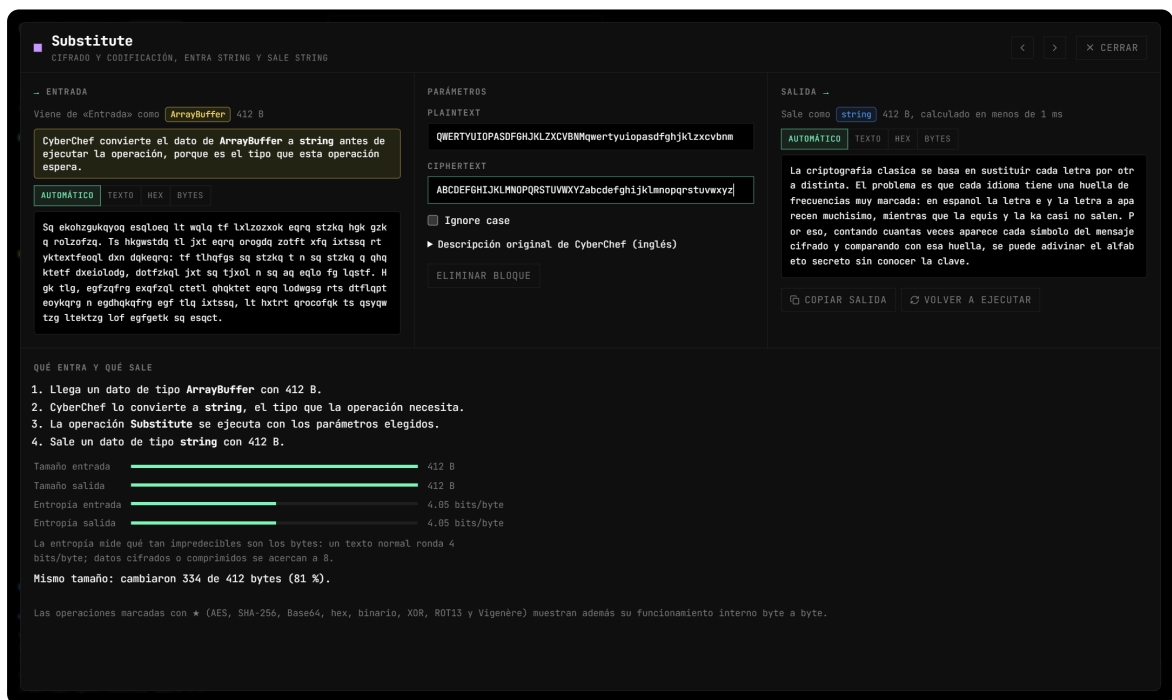
Con solo las dos primeras hipótesis (qtQT → aeAE) aparecen palabras sueltas como «a», «e» y «Es», y los huecos invitan a seguir (sa sezka pide ser «la letra»):



Ve añadiendo pares a medida que confirmas letras. La clave completa (que reconstruyes, no la conoces de antemano) es:

Plaintext (cifradas): QWERTYUIOPASDFGHJKLZXCVBNMqwertyuiopasdfghjklzxcvbnm
Ciphertext (en claro): ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz

Es decir, el cifrado usó el alfabeto del teclado QWERTY: A→Q, B→W, C→E...



Si pones las cadenas al revés (el alfabeto QWERTY en *Ciphertext*), Substitute **vuelve a cifrar** el texto en lugar de descifrarlo y la salida es otro galimatías: Lj tagimuxajngj tljsgtj...

El mensaje descifrado es:

«La criptografía clásica se basa en sustituir cada letra por otra distinta. El problema es que cada idioma tiene una huella de frecuencias muy marcada: en español la letra e y la letra a aparecen muchísimo...»

El propio texto describe la técnica que lo rompe.

Verifícalo con otras herramientas

Con `coreutils` basta (ver [requisitos](#)). Copia el texto de la *Entrada* a `cifrado.txt`, o extráelo desde la raíz del repositorio:

```
python3 -c "import json;s=open('src/labs.data.ts',encoding='utf8').read();d=json.loads(s[s.index('{'):s.rindex('}')+1]);open('cifrado.txt','w').write(d['sub']['cipher'])"
```

Frecuencia de cada letra, de mayor a menor (el equivalente de *Frequency distribution*, pero ordenado):

```
tr -cd 'a-zA-Z' < cifrado.txt | tr 'A-Z' 'a-z' | fold -w1 | sort | uniq -c | sort -rn | head -6
#      55 q
#      43 t
#      23 s
#      23 l
#      23 e
#      22 g
```

`tr` funciona igual que *Substitute*: cambia cada carácter del primer conjunto por el de la misma posición en el segundo. Por eso el orden es el mismo que en la app, **cifradas primero**:

```
tr 'qtQT' 'aeAE' < cifrado.txt # primeras hipótesis
# Sa ekohzgukayoa esaloea le wala ef lxlzoxok eara sezka hgk gzka rolzofza. Es hk...
tr 'QWERTYUIOPASDFGHJKLZXCVBNMqwertyuiopasdfghjklzxcvbnm' \
  'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz' < cifrado.txt
# La criptografía clásica se basa en sustituir cada letra por otra distinta. El problema...
```

Vigenère y el índice de coincidencia

La sustitución **polialfabética** (Vigenère) reparte varias sustituciones según una clave, lo que aplana el histograma y frena el análisis de frecuencias directo. Pero se rompe en dos pasos:

1. **Longitud de la clave** con el **índice de coincidencia** (bloque *Index of Coincidence*): se prueban distintos tamaños; cuando el texto se parte en columnas del tamaño correcto, cada columna es una sustitución simple y su IC se acerca al del idioma ($\approx 0,072$ en español) en vez del de texto aleatorio ($\approx 0,038$).
2. **Cada columna** se resuelve como un César con análisis de frecuencias, y el bloque *Vigenère Decode* aplica la clave hallada. (Puedes practicarlo con el ejemplo «Clásicos: ROT13 y Vigenère».)

Impacto real

Ninguno de estos cifrados ofrece seguridad hoy; su valor es didáctico e histórico (la máquina Enigma es, en el fondo, una sustitución polialfabética muy elaborada, y cayó por análisis estadístico y errores de operación). Enseñan el principio central: **un cifrado que preserva estructura estadística es vulnerable**.

Fíjate en el panel **Proceso** de *Substitute*: la entropía de entrada y de salida es la misma (4,05 bits/byte). La sustitución solo cambia los nombres de los símbolos; la distribución, que es lo que delata el idioma, no se mueve.

Mitigación

Los cifrados modernos producen salida indistinguible de aleatoria (difusión y confusión de Shannon): en CipherFlow, cifra un texto con AES y verás en el panel *Proceso* que su entropía se acerca a 8 bits/byte y el análisis de frecuencias se vuelve plano. Esa es la propiedad que le falta a la sustitución.

Lab 5 — Maleabilidad de CBC (bit-flipping)

Vulnerabilidad: el cifrado sin autenticar es *maleable*: un atacante puede modificar el texto cifrado para provocar cambios **predecibles** en el texto claro, sin conocer la clave. En CBC el descifrado del primer bloque es $P_0 = D(K, C_0) \oplus IV$. Como el IV se combina con XOR, voltear un bit del IV voltea el mismo bit del texto claro. Esta maleabilidad es la base de ataques mayores, como el **padding oracle** (POODLE, Lucky13, el fallo de ASP.NET de 2010).

El reto

El token cifrado descifra a `admin=0;alumno=Juan;grupo=fresas`. Consigue que descifre a `admin=1;...` **sin tocar la clave**.

LABORATORIO · AVANZADO

5 · Maleabilidad de CBC (bit-flipping)

CBC NO AUTENTICA: SE PUEDE ALTERAR EL TEXTO CLARO SIN LA CLAVE

OBJETIVO

Cambiar el mensaje descifrado sin conocer la clave, manipulando solo el IV.

EL RETO

El token cifrado descifra a «admin=0;...». Tu meta: que descifre a «admin=1;...» SIN tocar la clave. En CBC, $P_0 = D(C_0) \oplus IV$, así que al voltear un byte del IV volteas el mismo byte del texto claro. La rama de arriba calcula $IV \oplus \text{máscara}$ (la máscara ya tiene 0x01 en la posición 6). Copia ese «IV modificado» al parámetro IV del bloque «AES Decrypt» de abajo y observa el resultado.

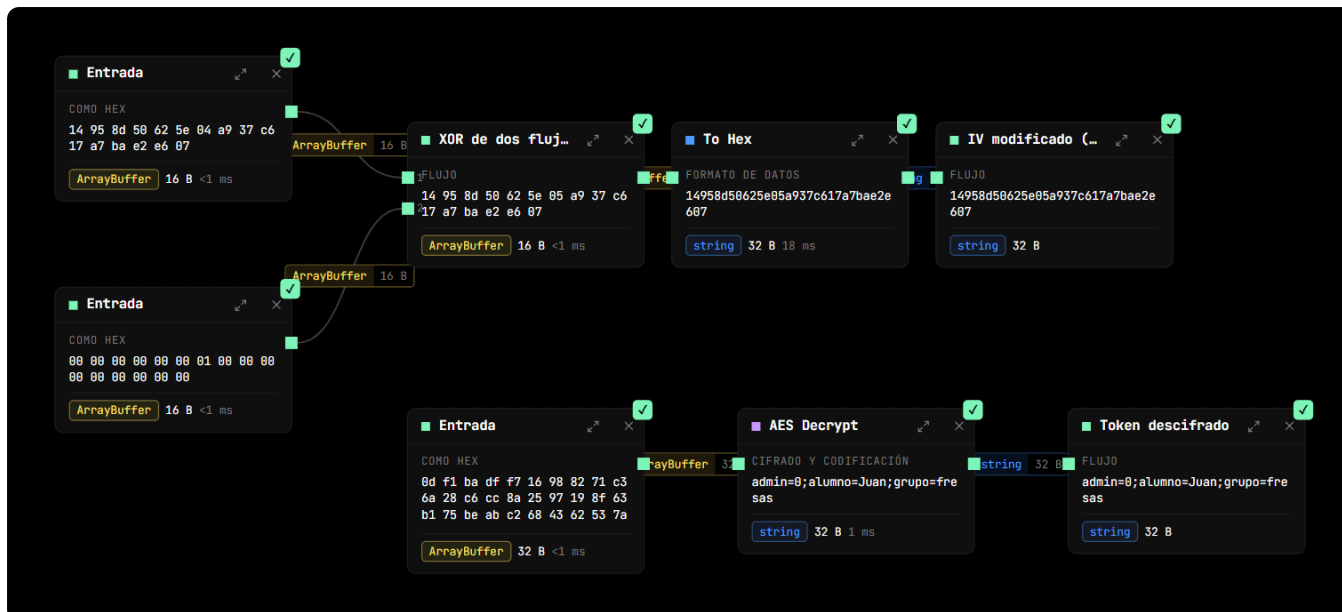
PISTAS (4)

LO LOGRASTE CUANDO...

La salida descifrada empieza por «admin=1».

VER SOLUCIÓN (WRITEUP)

AL LIENZO

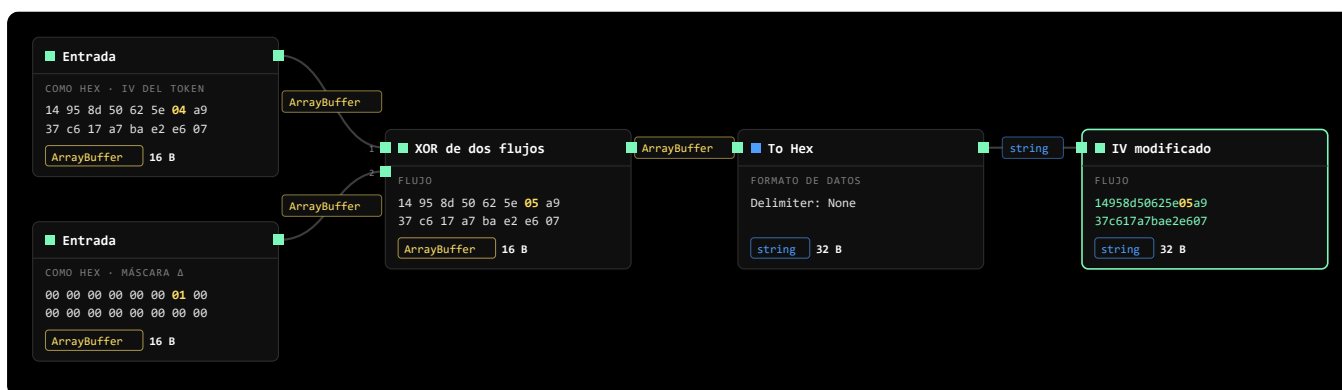


Solución

El objetivo es el byte en la posición 6 del texto claro: '0' (0x30) debe volverse '1' (0x31).

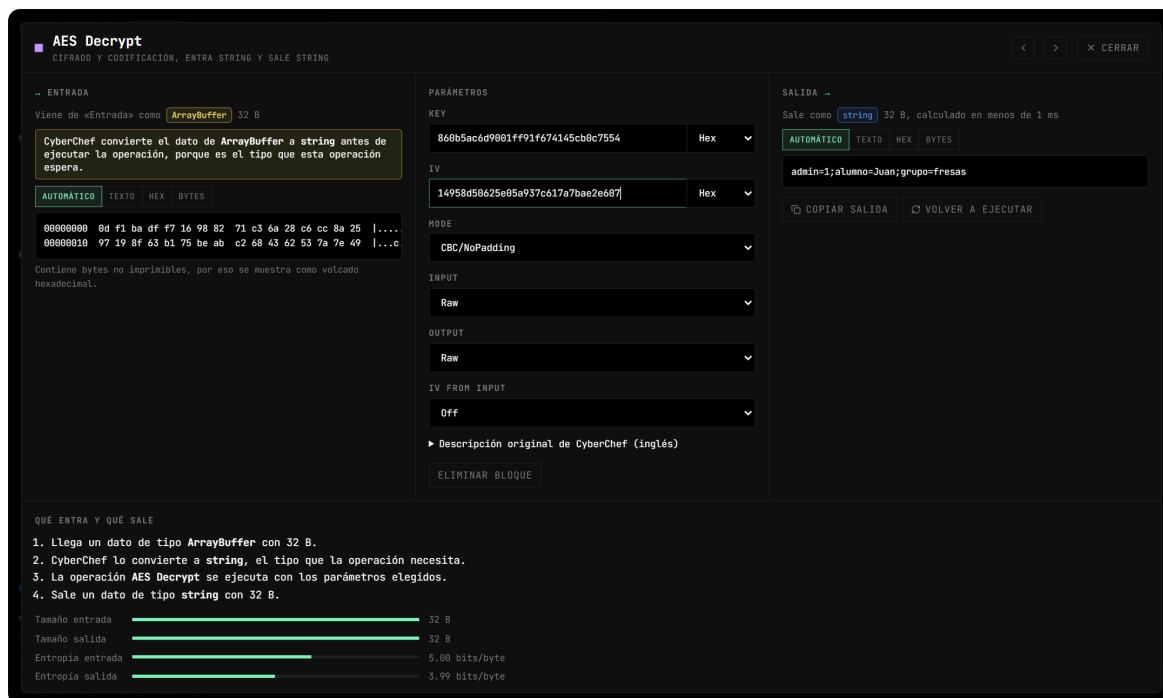
$$\Delta = '0' \oplus '1' = 0x30 \oplus 0x31 = 0x01$$

Como ese byte está en el primer bloque, se manipula el **IV** en esa misma posición. La rama superior del flujo ya lo calcula:



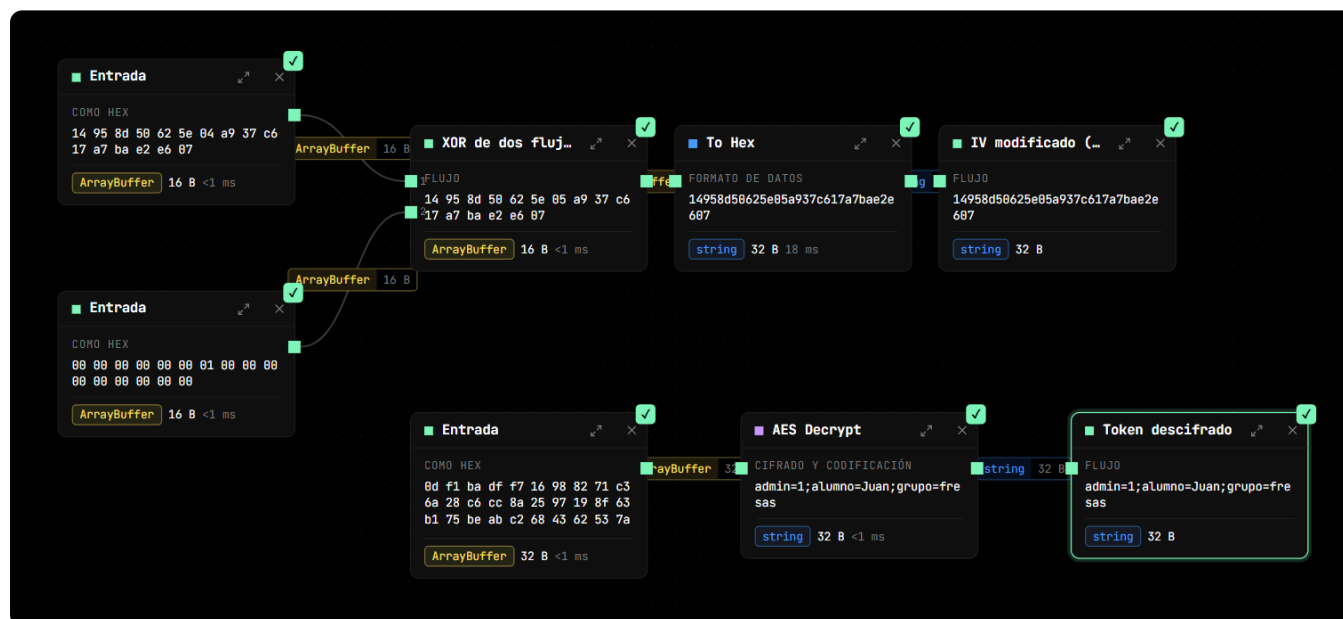
$IV' = IV \oplus \text{máscara}$ cambia solo el byte 6 del IV (resaltado): 04 pasa a 05. El resto del IV queda igual.

El paso del reto: copia el valor de *IV modificado* (botón **Copiar salida** en su detalle) y pégalo en el parámetro **IV** (formato Hex) del bloque *AES Decrypt* de abajo:



El resultado pasa a:

admin=1;alumno=Juan;grupo=fresas



Solo cambió `admin=0` → `admin=1`. El resto del mensaje queda intacto, porque el IV únicamente afecta al primer bloque.

El bloque de descifrado tiene la clave solo para que **veas** el efecto. El atacante real no la conoce ni la necesita: manipula el mensaje a ciegas, confiando en que sabe dónde está el byte objetivo.

Por qué

En CBC:

$$P_0 = D(K, C_0) \oplus IV$$
$$P'_0 = D(K, C_0) \oplus IV' = D(K, C_0) \oplus (IV \oplus \Delta) = P_0 \oplus \Delta$$

$D(K, C_0)$ no cambia porque ni la clave ni C_0 se tocan. El XOR con Δ se traslada directamente al texto claro:

CBC, primer bloque: $P_0 = D(K, C_0) \oplus IV$
Voltear un bit del IV voltea el mismo bit de P_0 . $D(K, C_0)$ no cambia: ni la clave ni C_0 se tocan.

POSICIÓN	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$D(K, C_0)$ lo fija el cifrado	75	f1	e0	39	0c	63	34	92	56	aa	62	ca	d4	8d	db	4d
IV original del token	14	95	8d	50	62	5e	04	a9	37	c6	17	a7	ba	e2	e6	07
P_0 texto claro	a	d	m	i	n	=	0	;	a	l	u	m	n	o	=	j

El atacante XORea el IV con la máscara $\Delta = '0' \oplus '1' = 0x01$ en la posición 6:

$D(K, C_0)$ igual que antes	75	f1	e0	39	0c	63	34	92	56	aa	62	ca	d4	8d	db	4d
$IV' = IV \oplus \Delta$ $04 \rightarrow 05$	14	95	8d	50	62	5e	05	a9	37	c6	17	a7	ba	e2	e6	07
P'_0 sin conocer K	a	d	m	i	n	=	1	;	a	l	u	m	n	o	=	j

$P'_0 = P_0 \oplus \Delta \rightarrow$ «admin=0» pasa a «admin=1»; los demás bytes quedan intactos.

Los valores son los reales del lab. En la posición 6, $D(K, C_0) = 0x34$; con el IV original $0x34 \oplus 0x04 = 0x30$ ('0'); con el IV modificado $0x34 \oplus 0x05 = 0x31$ ('1').

Si el byte objetivo estuviera en el bloque n , se voltearía el mismo byte de C_{n-1} en su lugar (a costa de convertir el bloque $n-1$ en basura, un compromiso habitual del ataque).

Verifícalo con otras herramientas

Con `openssl`, `xxd` y `bash` (ver [requisitos](#)). Descifra el token original:

```
K=860b5ac6d9001ff91f674145cb0c7554
IV=14958d50625e04a937c617a7bae2e607
echo 0df1badff716988271c36a28c6cc8a2597198f63b175beabc2684362537a7e49 | xxd -r -p > token.bin
openssl enc -d -aes-128-cbc -K $K -iv $IV -nopad -in token.bin
# admin=0;alumno=Juan;grupo=fresas
```

Calcula $IV' = IV \oplus \Delta$ en el byte 6 (caracteres 12–13 de la cadena hex) y descifra con él:

```

IV2=$(printf '%s%02x%s' ${IV:0:12} $((0x${IV:12:2} ^ 0x01)) ${IV:14})
echo $IV2
# 14958d50625e05a937c617a7bae2e607
openssl enc -d -aes-128-cbc -K $K -iv $IV2 -nopad -in token.bin
# admin=1;alumno=Juan;grupo=fresas

```

¿Por qué `-nopad`? El token son exactamente 32 bytes sin relleno PKCS#7 (en CipherFlow, modo CBC/NoPadding). Si lo quitas, `openssl` busca un relleno válido al final, no lo encuentra y responde `bad decrypt`. Justo esa diferencia entre «relleno válido» y «relleno inválido», si un servidor la deja ver, es el oráculo del ataque de la sección siguiente.

Reto extra: cambiar el segundo bloque. Para tocar bytes del bloque 1 hay que voltear el mismo byte de `C0`. Así se cambia `grupo=fresas` por `grupo=mangos` (posiciones 26–31, es decir, 10–15 dentro del bloque 1):

```

python3 -c "c=bytearray(open('token.bin','rb').read());d=bytes(a^b for a,b in zip(b'fresas',b'mangos'));c[10:16]=bytes(x^y for x,y in zip(c[10:16],d));open('token2.bin','wb').write(c)"
openssl enc -d -aes-128-cbc -K $K -iv $IV -nopad -in token2.bin | xxd
# 00000000: 0d2e 5898 d49d c2ae 7f47 ddfb 4a1c 232f  ..X.....G..J.#/    ← bloque 0 destruido
# 00000010: 7561 6e3b 6772 7570 6f3d 6d61 6e67 6f73  uan;grupo=mangos    ← bloque 1 a la carta

```

El bloque 1 queda exactamente como quería el atacante, pero el bloque 0 se convierte en basura: ese es el compromiso del que habla la sección anterior.

La mitigación, comprobada. `openssl enc` no admite modos AEAD (con `-aes-128-gcm` responde `AEAD ciphers not supported`), así que usa Node, que ya necesitas para CipherFlow. Guarda esto como `gcm.js` y ejecútalo con `node gcm.js`:

```

const crypto = require('node:crypto')
const key = Buffer.from('860b5ac6d9001ff91f674145cb0c7554', 'hex'), iv = crypto.randomBytes(12)
const enc = crypto.createCipheriv('aes-128-gcm', key, iv)
const ct = Buffer.concat([enc.update('admin=0;alumno=Juan;grupo=fresas'), enc.final()] ), tag = enc.getAuthTag()
const abrir = c => { const d = crypto.createDecipheriv('aes-128-gcm', key, iv); d.setAuthTag(tag); return Buffer.concat([d.update(c), d.final()]).toString() }
console.log('original :', abrir(ct))
const mod = Buffer.from(ct); mod[6] ^= 0x01 // el mismo volteo de bit que en CBC
try { console.log('alterado :', abrir(mod)) } catch (e) { console.log('alterado :', e.message) }

```

```

original : admin=0;alumno=Juan;grupo=fresas
alterado : Unsupported state or unable to authenticate data

```

GCM cifra en modo contador, así que el volteo sí cambiaría `'0'` por `'1'` en el texto claro, pero la etiqueta ya no coincide y el mensaje se rechaza antes de entregarlo.

Del bit-flipping al padding oracle

Este lab muestra el *primitivo*. El **padding oracle** lo convierte en un descifrado completo: si el servidor revela (por un mensaje de error o por el tiempo de respuesta) si el relleno PKCS#7 de un mensaje es válido, el atacante ajusta byte a byte el bloque anterior hasta forzar un relleno válido y, de ahí, deduce $D(K, C_i)$ y por tanto el texto claro —sin la clave—. Es adaptativo (cientos de consultas por bloque), por eso no se resuelve en un grafo estático como este; pero descansa exactamente en la maleabilidad que acabas de explotar.

Impacto real

- **POODLE (2014)**: padding oracle sobre SSL 3.0 con CBC.
- **Lucky Thirteen (2013)**: oracle por tiempo en TLS.
- **ASP.NET (2010)**: padding oracle que permitía leer archivos y forjar cookies.
- **Cookies de sesión** cifradas con CBC sin MAC: bit-flipping para escalar privilegios, justo como este

`admin=0` → `admin=1`.

Mitigación

Usa **cifrado autenticado (AEAD)**: AES-GCM o ChaCha20-Poly1305. Añaden una etiqueta que se verifica antes de descifrar, así que cualquier bit alterado se detecta y el mensaje se rechaza. Pruébalo en CipherFlow: cifra un texto con *AES Encrypt* en modo **GCM** (Output `Hex`); la salida trae el cifrado y, debajo, una línea `Tag: ...`. Pasa ese cifrado a *AES Decrypt* en modo GCM (Input `Hex`) con el tag en el parámetro **GCM Tag** y descifra bien. Ahora cambia un solo dígito del cifrado o del IV: en vez de entregar un texto claro manipulado, el bloque falla con *Unable to decrypt input with these parameters*. Si no puedes usar AEAD, aplica **encrypt-then-MAC** con un MAC de tiempo constante.